

第3章 ファイルを修復してみよう

この章で学ぶこと

- ファイルシグネチャ
- hex
- xxd

1章ではfileコマンドでファイルの正体を見抜き、ファイル先頭のバイト列（ファイルシグネチャ）を読んで判定していることを学びました。

この章ではxxdというコマンドを使って、そのバイト列を自分の目で確認し、壊れたファイルを自分の手で修復することに挑戦してみましょう。

16進数とは

`xxd` の出力を理解するために、まず**16進数**について軽く触れます。

普段私たちが使っている数字は**10進数**です。0～9の10種類の数字で数を表します。

コンピュータの世界でよく使われる**16進数**は、0～9に加えてA～Fを使い、16種類の文字で数を表します。

xxdの使い方

xxdは対象ファイルの中身を16進数のバイト列として表示するコマンドです。

例： `xxd <対象のファイル名>`

例として、適当なテキストファイルに対して実行してみましょう。

```
~/workshop master  
> echo "hello" > test.txt  
  
~/workshop master  
> xxd test.txt  
00000000: 6865 6c6c 6f0a                hello.
```

出力は3つの列に分かれています。

オフセット (00000000) : ファイルの先頭からの位置 (16進数表記)

バイト列 (6865...6f0a) : 実際のデータを16進数で表示

ASCII (hello.) : 印字可能な文字はそのまま表示し、それ以外は.を表示

hは68、eは65、lは6c、oは6fに対応しています。

PNGファイルを修復してみよう

dog.pngというファイルを用意しましたが、以下のように開けません。

xxdでファイルを確認してみましょう。



xxdで先頭のバイト列を確認してみると以下ようになります。

headは先頭からデフォルトで10行だけ確認するコマンドです。

```
~/Downloads master
> xxd dog.png | head
00000000: dead beef 0d0a 1a0a 0000 000d 4948 4452 .....IHDR
00000010: 0000 0190 0000 00c8 0802 0000 0049 7720 .....Iw
00000020: b500 0015 6349 4441 5478 9ced dd7b 5094 ....cIDATx...{P.
00000030: e7bd 07f0 dffb ee8d 5d64 1710 96fb ee02 .....]d.....
00000040: 0a01 1414 a378 e1a2 c6d2 5cd4 9e68 63db .....x....\..hc.
00000050: d812 d349 3339 49cf 744c 7a99 73d2 b44d ...I39I.tLz.s..M
00000060: 33ed e99c d6a4 693a 4edb 9c34 4dd2 9ca6 3.....i:N..4M...
00000070: eae4 0a9a 6a54 1034 8880 7255 91cb 0272 ....jT.4..rU...r
00000080: 7117 76b9 5f76 d9f7 3d7f bcba d92e 2b22 q.v._v..=.....+
00000090: a0f8 d4ef 67fc 63f7 dde7 faee ec97 e779 ....g.c.....y
```

通常のpngのファイルシグネチャはファイルの先頭から次のようなバイト列はずです。

89 50 4E 47 0D 0A 1A 0A	%PNG ^C _R ^L _F ^{SUB} _L ^F	0	png	Ima the Net forr Net
----------------------------	---	---	-----	----------------------------------

現状：dead beef 0d0a 1a0a

通常：8950 4e47 0d0a 1a0a

のようになっています。なので、通常状態に戻してあげましょう。

いろんな方法がありますが、Pythonを利用して直していきます。

```
1 with open("dog.png", 'r+b') as f:
2     f.seek(0x00)
3     f.write(b'\x89\x50\x4e\x47')
```

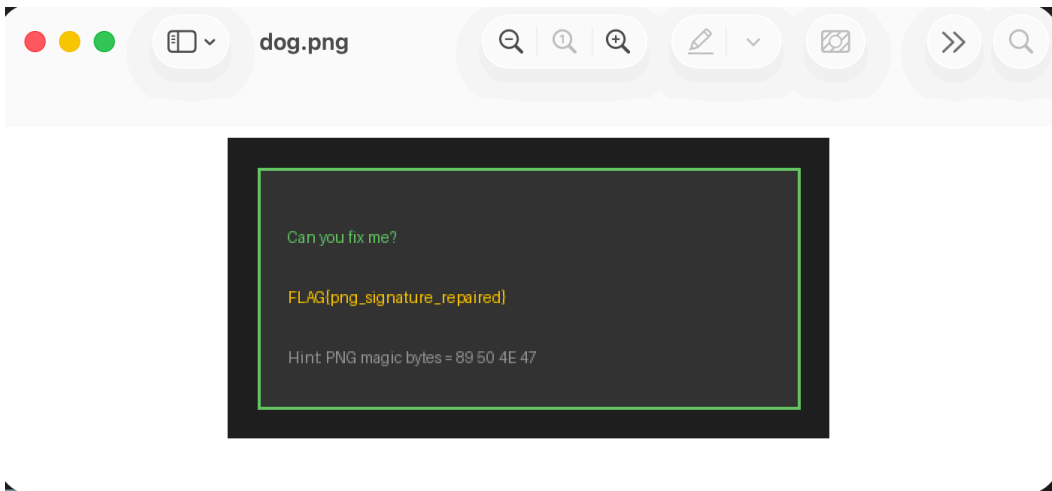
▼ コードの意味

dog.pngというファイルをバイト列を読むモードで開き、fという名前で保持する
書きたい場所は今回ファイルの先頭行である0x00000000なので
seekという関数で0x00000000に移動する。
最後に、writeで書き換えたいバイト列を書き込む。

上のPythonコードを実行し、もう一度xxdを行うと、次のようになります。

```
~/Downloads master
) xxd dog.png | head
00000000: 8950 4e47 0d0a 1a0a 0000 000d 4948 4452 .PNG.....IHDR
00000010: 0000 0190 0000 00c8 0802 0000 0049 7720 .....Iw
00000020: b500 0015 6349 4441 5478 9ced dd7b 5094 ....cIDATx...{P.
00000030: e7bd 07f0 dffb ee8d 5d64 1710 96fb ee02 .....]d.....
00000040: 0a01 1414 a378 e1a2 c6d2 5cd4 9e68 63db .....x....\..hc.
00000050: d812 d349 3339 49cf 744c 7a99 73d2 b44d ...I39I.tLz.s..M
00000060: 33ed e99c d6a4 693a 4edb 9c34 4dd2 9ca6 3.....i:N..4M...
00000070: eae4 0a9a 6a54 1034 8880 7255 91cb 0272 ....jT.4..rU...r
00000080: 7117 76b9 5f76 d9f7 3d7f bcba d92e 2b22 q.v._v..=.....+
00000090: a0f8 d4ef 67fc 63f7 dde7 faee ec97 e779 ....g.c.....y
```

ファイルシグネチャを直すと開けるようになりました。



まとめ

16進数とは0～9に加えA～Fの計16種類で数を表す記数法。コンピュータの世界でよく使われる。

`xxd` とはファイルの中身を**16進数のバイト列**として表示するコマンド

`xxd <ファイル名>`

出力は3列構成：

列	内容
オフセット	ファイル先頭からの位置（16進数）
バイト列	実際のデータ（16進数）
ASCII	印字可能な文字はそのまま、それ以外は <code>.</code>

ファイルが開けない場合、**ファイルシグネチャが壊れている**可能性があります。

例：dog.pngの場合

```
現状：dead beef 0d0a 1a0a
正常：8950 4e47 0d0a 1a0a
```

Pythonで修復できます（他の方法も実はある）：

1. バイナリ読み書きモードでファイルを開く
2. `seek()` で修正したい位置（先頭=`0x00`）に移動
3. `write()` で正しいバイト列を書き込む

以下の課題に取り組んでみてください。

問題6：PDFのファイルシグネチャはなんですか？調べてみてください。

問題7：3のディレクトリに含まれるdog.pdfは壊れています。修復してみてください。（fileしてPDFと出たらクリア）

難-問題8：3のディレクトリに含まれるproblemは壊れています。元々なんの形式のファイルだったのでしょうか？また、FLAG{}形式で情報が隠されているので探してみてください。